Set 1

-- Q1: Create an Employee Attendance table and use UPDATE queries to modify attendance based on conditions. Show final results with SELECT.

```
CREATE TABLE EmployeeAttendance (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    DaysPresent INT
);

INSERT INTO EmployeeAttendance VALUES
(1, 'Alice', 20),
(2, 'Bob', 15),
(3, 'Charlie', 10),
(4, 'Diana', 25);

UPDATE EmployeeAttendance
SET DaysPresent = DaysPresent + 2
WHERE DaysPresent < 15;

SELECT * FROM EmployeeAttendance;
```

-- Q2: Create a Product table with constraints like UNIQUE and CHECK on price. Insert values to validate the constraints work correctly.

```
CREATE TABLE Product (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50) UNIQUE,
    Price DECIMAL(10,2) CHECK (Price > 0)
);

INSERT INTO Product VALUES
(101, 'Keyboard', 25.00),
(102, 'Mouse', 15.50),
(103, 'Monitor', 150.00);
```

-- Q3: Use RIGHT JOIN on Customers and Orders to display all orders including those with unknown customers.

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50)
);

CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerID INT,
    OrderDate DATE
);

INSERT INTO Customers VALUES
(1, 'John Doe'),
(2, 'Jane Smith');

INSERT INTO Orders VALUES
(101, 1, '2024-04-01'),
(102, 3, '2024-04-02'), -- unknown customer
(103, 2, '2024-04-03');

SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate
FROM Customers
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Set 2

-- Q1 (15 marks): Create Orders and Customers tables and retrieve customer order history using different types of JOINS.

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50)
);
```

```
CREATE TABLE Orders (
   OrderID INT PRIMARY KEY,
   CustomerID INT,
   OrderDate DATE
);

INSERT INTO Customers VALUES
(1, 'Alice'),
(2, 'Bob'),
(3, 'Charlie');

INSERT INTO Orders VALUES
(101, 1, '2024-01-01'),
(102, 2, '2024-02-01'),
(103, NULL, '2024-03-01');

SELECT Customers.CustomerName, Orders.OrderID, Orders.OrderDate
FROM Customers
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;

SELECT Customers.CustomerName, Orders.OrderID, Orders.OrderDate
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;

SELECT Customers.CustomerName, Orders.OrderID, Orders.OrderDate
FROM Customers
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

-- Q2 (15 marks): Build a SalesRecord table and apply aggregate functions to summarize sales by region or category.

```
CREATE TABLE SalesRecord (
   SaleID INT PRIMARY KEY,
   Region VARCHAR(50),
   Category VARCHAR(50),
```

```
    Amount DECIMAL(10,2)
);

INSERT INTO SalesRecord VALUES
(1, 'North', 'Electronics', 1000.00),
(2, 'South', 'Clothing', 500.00),
(3, 'North', 'Clothing', 800.00),
(4, 'West', 'Electronics', 1200.00);

SELECT Region, SUM(Amount) AS TotalSales
FROM SalesRecord
GROUP BY Region;

SELECT Category, AVG(Amount) AS AvgSales
FROM SalesRecord
GROUP BY Category;
```

Q3 (10 marks): Perform UPDATE on a sales table to increase all sales values by 10% and display results

```
CREATE TABLE SalesRecord (
    SaleID INT PRIMARY KEY,
    Region VARCHAR(50),
    Category VARCHAR(50),
    Amount DECIMAL(10,2)
);
INSERT INTO SalesRecord VALUES
(1, 'North', 'Electronics', 1000.00),
(2, 'South', 'Clothing', 500.00),
(3, 'East', 'Groceries', 300.00),
(4, 'West', 'Electronics', 1200.00);

UPDATE SalesRecord
SET Amount = Amount * 1.10;

SELECT * FROM SalesRecord;
```

Set3

-- Q1 (15 marks): Create a Books table with various columns, insert data, and use SELECT queries with different clauses (WHERE, ORDER BY, LIMIT).

```
CREATE TABLE Books (
    BookID INT PRIMARY KEY,
    Title VARCHAR(100),
    Author VARCHAR(50),
    Price DECIMAL(6,2),
    PublishYear INT
);
INSERT INTO Books VALUES
(1, 'SQL Essentials', 'John Doe', 29.99, 2020),
(2, 'Learn Python', 'Jane Roe', 35.50, 2021),
(3, 'Data Science 101', 'Alan Smith', 40.00, 2019),
(4, 'Intro to Databases', 'Emily Clark', 25.00, 2022),
(5, 'Advanced SQL', 'David Kent', 55.00, 2018);

SELECT * FROM Books WHERE Price > 30;

SELECT * FROM Books ORDER BY Price DESC;

SELECT * FROM Books ORDER BY Price ASC LIMIT 3;
```

-- Q2 (15 marks): Create an Employee Attendance table and use UPDATE queries to modify attendance based on conditions. Show final results with SELECT.

```
CREATE TABLE EmployeeAttendance (
    EmployeeID INT PRIMARY KEY,
    EmployeeName VARCHAR(50),
    DaysPresent INT
);
INSERT INTO EmployeeAttendance VALUES
(1, 'Anil', 12),
(2, 'Rekha', 18),
```

```
(3, 'Karan', 9),
(4, 'Deepa', 21);

UPDATE EmployeeAttendance
SET DaysPresent = DaysPresent + 3
WHERE DaysPresent < 15;

SELECT * FROM EmployeeAttendance;

-- Q3 (10 marks): Apply SUM and COUNT on a table storing product prices and quantities.
CREATE TABLE ProductInventory (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Price DECIMAL(8,2),
    Quantity INT
);
INSERT INTO ProductInventory VALUES
(1, 'Pen', 10.00, 100),
(2, 'Notebook', 25.00, 50),
(3, 'Stapler', 45.00, 30);

SELECT SUM(Price * Quantity) AS TotalSalesValue FROM ProductInventory;

SELECT COUNT(*) AS TotalProducts FROM ProductInventory;
```

Set4

```
-- Q1 (15 marks): Design a database and create tables to manage department and employee details.
CREATE TABLE Departments (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);

CREATE TABLE Employees (
```

```
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DeptID INT,
    Salary DECIMAL(10,2),
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);
INSERT INTO Departments VALUES
(1, 'HR'),
(2, 'IT'),
(3, 'Finance');

INSERT INTO Employees VALUES
(101, 'Amit', 1, 50000),
(102, 'Sneha', 2, 60000);

UPDATE Employees
SET DeptID = 2
WHERE EmpID = 101;

-- Q2 (15 marks): Create a comprehensive table with Employee data using multiple data types.
CREATE TABLE EmployeeDetails (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    DOB DATE,
    JoinDate DATE,
    Salary DECIMAL(10,2),
    Position VARCHAR(50)
);
INSERT INTO EmployeeDetails VALUES
(1, 'Ravi', '1990-05-12', '2015-06-01', 55000.00, 'Manager'),
(2, 'Priya', '1992-08-22', '2017-09-15', 45000.00, 'Analyst'),
(3, 'Vikram', '1988-03-18', '2012-11-20', 70000.00, 'Director');

UPDATE EmployeeDetails
```

```
SET Salary = 50000
WHERE EmpName = 'Priya';

DELETE FROM EmployeeDetails
WHERE Salary > 65000;

-- Q3 (10 marks): Insert and delete records from an Employee table and display all entries using SELECT.
INSERT INTO EmployeeDetails VALUES
(4, 'Anjali', '1994-12-05', '2020-01-01', 40000.00, 'HR Executive');

DELETE FROM EmployeeDetails
WHERE Salary < 45000;

SELECT * FROM EmployeeDetails;
```

Set5

```
-- Q1 (15 marks): Create a student record system with columns for StudentID, Name, Marks, and Grade.
CREATE TABLE StudentRecords (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Marks INT,
    Grade CHAR(1)
);
INSERT INTO StudentRecords VALUES
(1, 'Neha', 85, 'A'),
(2, 'Arjun', 67, 'B'),
(3, 'Mira', 42, 'C'),
(4, 'Rohan', 35, 'D');

SELECT * FROM StudentRecords WHERE Marks > 60;

SELECT * FROM StudentRecords WHERE Grade IN ('A', 'B');
```

-- Q2 (15 marks): Create a table with a composite primary key and demonstrate how it enforces uniqueness across combined columns.

```
CREATE TABLE Enrollments (
   StudentID INT,
   CourseID INT,
   EnrollmentDate DATE,
   PRIMARY KEY (StudentID, CourseID)
);
INSERT INTO Enrollments VALUES (1, 101, '2024-01-10');
INSERT INTO Enrollments VALUES (1, 102, '2024-01-15');
INSERT INTO Enrollments VALUES (2, 101, '2024-01-20');
```

-- Q3 (10 marks): Demonstrate LEFT JOIN on Departments and Employees tables to list all departments with or without employees.

```
CREATE TABLE Departments2 (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50)
);
CREATE TABLE Employees2 (
   EmpID INT PRIMARY KEY,
   EmpName VARCHAR(50),
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES Departments2(DeptID)
);
INSERT INTO Departments2 VALUES
(1, 'HR'),
(2, 'IT'),
(3, 'Finance');

INSERT INTO Employees2 VALUES
(201, 'Lakshmi', 1),
(202, 'Nikhil', 2);
SELECT Departments2.DeptName, Employees2.EmpName
FROM Departments2
LEFT JOIN Employees2 ON Departments2.DeptID = Employees2.Dept
```

Set6

-- Q1 (15 marks): Create and drop a table using DDL. Insert, update, delete data using DML. Show SELECT queries after each operation to verify.

```
CREATE TABLE TempEmployees (
    EmpID INT PRIMARY KEY,
    EmpName VARCHAR(50),
    Salary DECIMAL(10,2)
);
INSERT INTO TempEmployees VALUES
(1, 'Rahul', 45000),
(2, 'Meena', 55000),
(3, 'Akash', 40000);

SELECT * FROM TempEmployees;

UPDATE TempEmployees
SET Salary = Salary + 5000
WHERE EmpID = 2;

SELECT * FROM TempEmployees;

DELETE FROM TempEmployees
WHERE EmpID = 3;

SELECT * FROM TempEmployees;

DROP TABLE TempEmployees;
```

-- Q2 (15 marks): Create a Product table with constraints like UNIQUE and CHECK on price.

```
CREATE TABLE ProductWithConstraints (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50) UNIQUE,
    Price DECIMAL(8,2) CHECK (Price > 0)
```

```
);
INSERT INTO ProductWithConstraints VALUES
(1, 'USB Drive', 20.00),
(2, 'Webcam', 50.00);

SELECT * FROM ProductWithConstraints;
```

-- Q3 (10 marks): Test CHECK constraint on a Grade column in a student table by inserting valid/invalid grades.

```
CREATE TABLE StudentGrades (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Grade CHAR(1) CHECK (Grade IN ('A', 'B', 'C', 'D', 'F'))
);

INSERT INTO StudentGrades VALUES (1, 'Tanvi', 'B');

SELECT * FROM StudentGrades;
```

Set7

-- Q1 (15 marks): Build a database with EmployeeID, Name, Salary, Department, and ActiveStatus.

```
CREATE TABLE Employees (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10,2),
    Department VARCHAR(50),
    ActiveStatus BOOLEAN
);
INSERT INTO Employees VALUES
(1, 'John', 50000, 'IT', TRUE),
(2, 'Sarah', 60000, 'HR', FALSE),
(3, 'David', 45000, 'IT', TRUE),
(4, 'Emma', 55000, 'Finance', TRUE);
```

```
SELECT * FROM Employees WHERE Salary > 50000;


SELECT * FROM Employees WHERE ActiveStatus = TRUE;


-- Q2 (15 marks): Create a table using a primary key and a foreign key referencing another table.
CREATE TABLE Departments (
    DeptID INT PRIMARY KEY,
    DeptName VARCHAR(50)
);
INSERT INTO Departments VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');


CREATE TABLE EmployeesWithDept (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    DeptID INT,
    Salary DECIMAL(10,2),
    FOREIGN KEY (DeptID) REFERENCES Departments(DeptID)
);


INSERT INTO EmployeesWithDept VALUES
(1, 'John', 1, 50000),  -- Valid DeptID 1 (IT)
(2, 'Sarah', 2, 60000);  -- Valid DeptID 2 (HR)


-- Q3 (10 marks): Show usage of DEFAULT value by inserting new records into a table without specifying all fields.
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Category VARCHAR(50) DEFAULT 'General',
    StockQuantity INT DEFAULT 0
);
```

```
INSERT INTO Products (ProductID, ProductName) VALUES
(1, 'Laptop'),
(2, 'Smartphone');

SELECT * FROM Products;
```

Set8

```
-- Q1 (15 marks): Create a table with a CHECK constraint on the Age column.
CREATE TABLE EmployeesWithAge (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    Age INT CHECK (Age >= 18 AND Age <= 100)  -- Age must be between 18 and 100
);
INSERT INTO EmployeesWithAge VALUES
(1, 'John', 30),
(2, 'Sarah', 25);

 INSERT INTO EmployeesWithAge VALUES (3, 'David', 15);
 INSERT INTO EmployeesWithAge VALUES (4, 'Emma', 105);

SELECT * FROM EmployeesWithAge;

-- Q2 (15 marks): Create a table with NOT NULL and DEFAULT constraints and verify them using sample INSERT statements.
CREATE TABLE Products (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50) NOT NULL,  -- Cannot be NULL
    Category VARCHAR(50) DEFAULT 'General',  -- Default value 'General'
    Price DECIMAL(10, 2) NOT NULL,  -- Cannot be NULL
    StockQuantity INT DEFAULT 0  -- Default value 0
);

INSERT INTO Products (ProductID, ProductName, Price) VALUES
(1, 'Laptop', 800.00);
```

```
INSERT INTO Products (ProductID, ProductName, Category, Price, StockQuantity) VALUES
(2, 'Smartphone', 'Electronics', 500.00, 100);

SELECT * FROM Products;

-- Q3 (10 marks): Use SELECT queries to retrieve the top 3 employees with the highest salaries.
CREATE TABLE EmployeesSalaries (
    EmployeeID INT PRIMARY KEY,
    Name VARCHAR(50),
    Salary DECIMAL(10,2)
);
INSERT INTO EmployeesSalaries VALUES
(1, 'John', 50000),
(2, 'Sarah', 60000),
(3, 'David', 70000),
(4, 'Emma', 80000),
(5, 'Ravi', 95000);

SELECT * FROM EmployeesSalaries
ORDER BY Salary DESC
LIMIT 3;
```

Set9

```
-- Q1 (15 marks): Create a sales table with Product Name, Quantity Sold, and UnitPrice.
CREATE TABLE Sales (
    SalesID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    QuantitySold INT,
    UnitPrice DECIMAL(10, 2)
);
INSERT INTO Sales VALUES
(1, 'Laptop', 10, 800.00),
```

```
(2, 'Smartphone', 20, 500.00),
(3, 'Tablet', 15, 300.00),
(4, 'Headphones', 25, 50.00);

SELECT ProductName,
    SUM(QuantitySold * UnitPrice) AS TotalSales,
    AVG(QuantitySold * UnitPrice) AS AverageSales
FROM Sales
GROUP BY ProductName;
```

-- Q2 (15 marks): Create a table of customers and orders with foreign key relationships and show INNER JOIN results.

```
CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  CustomerName VARCHAR(50)
);

CREATE TABLE Orders (
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  OrderDate DATE,
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

INSERT INTO Customers VALUES
(1, 'John Doe'),
(2, 'Jane Smith'),
(3, 'Tom Harris');

INSERT INTO Orders VALUES
(101, 1, '2025-01-10'),
(102, 2, '2025-02-15'),
(103, 1, '2025-03-05');

SELECT Customers.CustomerID, Customers.CustomerName, Orders.OrderID, Orders.OrderDate
```

```
FROM Customers
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

-- Q3 (10 marks): Perform DELETE operations on rows in a customer table based on a condition and show the result.

```
CREATE TABLE Customers (
    CustomerID INT PRIMARY KEY,
    CustomerName VARCHAR(50),
    Email VARCHAR(100),
    PhoneNumber VARCHAR(20)
);

INSERT INTO Customers (CustomerID, CustomerName, Email, PhoneNumber) VALUES
(1, 'John Doe', 'john.doe@email.com', '123-456-7890'),
(2, 'Jane Smith', 'jane.smith@email.com', '234-567-8901'),
(3, 'Tom Harris', 'tom.harris@email.com', '345-678-9012'),
(4, 'Emma Brown', 'emma.brown@email.com', '456-789-0123');

SELECT * FROM Customers;

DELETE FROM Customers
WHERE CustomerID = 3;

SELECT * FROM Customers;
```

Set10

-- Q1 (15 marks): Create a Movies table and perform queries to update movie ratings and delete movies based on release year.

```
CREATE TABLE Movies (
    MovieID INT PRIMARY KEY,
    MovieName VARCHAR(100),
    ReleaseYear INT,
    Rating DECIMAL(3,1)  -- Movie rating with one decimal point
);
```

```
INSERT INTO Movies (MovieID, MovieName, ReleaseYear, Rating) VALUES
(1, 'The Shawshank Redemption', 1994, 9.3),
(2, 'The Dark Knight', 2008, 9.0),
(3, 'Inception', 2010, 8.8),
(4, 'The Matrix', 1999, 8.7),
(5, 'Interstellar', 2014, 8.6);

UPDATE Movies
SET Rating = 9.0
WHERE MovieName = 'Inception';

DELETE FROM Movies
WHERE ReleaseYear < 2000;

SELECT * FROM Movies;
```

-- Q2 (15 marks): Create an Inventory table with CHECK constraint on quantity and use SELECT to display products with low stock.

```
CREATE TABLE Inventory (
    ProductID INT PRIMARY KEY,
    ProductName VARCHAR(50),
    Quantity INT CHECK (Quantity >= 0),  -- Ensure that Quantity is not negative
    Price DECIMAL(10,2)
);

INSERT INTO Inventory (ProductID, ProductName, Quantity, Price) VALUES
(1, 'Laptop', 10, 800.00),
(2, 'Smartphone', 5, 500.00),
(3, 'Tablet', 0, 300.00),  -- Low stock item (Quantity = 0)
(4, 'Headphones', 15, 50.00);

SELECT * FROM Inventory
WHERE Quantity < 5;
```

-- Q3 (10 marks): Update marks in a student table where the score is below 40. Show updated table.

```
CREATE TABLE Students (
    StudentID INT PRIMARY KEY,
    Name VARCHAR(50),
    Marks INT
);

INSERT INTO Students (StudentID, Name, Marks) VALUES
(1, 'John Doe', 55),
(2, 'Jane Smith', 35),  -- Below 40 (this should be updated)
(3, 'Tom Harris', 45),
(4, 'Emma Brown', 25);  -- Below 40 (this should be updated)

UPDATE Students
SET Marks = 40
WHERE Marks < 40;

SELECT * FROM Students;
```